CSA04-Operating Systems

TASK 7 – Evaluate Process Control System Calls

Name: V. Reddy Sreya Regno:192525124

1. Process Control System Calls

A command shell on a database server manages programs using process control system calls. These calls help create, execute, wait for and terminate processes.

Functions of system calls

1. fork(): Creates a new child process by duplicating the calling process.
2. exec(): Replaces the current process with a new program.
3. wait(): Makes the parent process wait until a child process finishes.
4. exit(): Terminates a process and returns an exit status to the operating system.

2. How a shell runs a command

- The shell receives a command from the user.
- It calls fork() to create a child process.
- The child uses exec() to execute the requested command.
- The parent uses wait() or waitpid() to wait for the child.
- After completion, the shell obtains the child's exit status and displays the result.

3. Zombie and Orphan Processes

Zombie Process:

- A child process that has completed execution but whose exit status has not yet been collected by its parent.
- It remains in the process table.
- The OS removes it after the parent calls wait() or waitpid().

Orphan Process:

- A child process whose parent terminates before it finishes.
- The OS assigns it to a system reaper process (commonly PID 1 or a designated subreaper).
- It is eventually collected when it terminates.

4. Why fork() followed by exec() is used

- fork() creates a separate child process.
- exec() allows the child to run a different program.
- The parent can continue running independently.
- It provides better process control and isolation.
- It allows the shell to configure input, output and redirection before executing a command.

5. Complete the Code

The following C program creates two child processes. The first executes the Linux expr command, and the second exits with status 3.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
   pid_t pid;
   int status;

   printf("shell: running job 1\n");
   fflush(stdout);

   // TODO 1: Create first child process
   pid = fork();

   if (pid < 0) {
      perror("fork");
      return 1;
   }

   if (pid == 0) {
      execlp("expr", "expr", "6", "*", "7", (char *)NULL);
      _exit(127);
   }

   waitpid(pid, &status, 0);

   if (WIFEXITED(status))
      printf("job 1 exit status %d\n", WEXITSTATUS(status));
```

```
    printf("shell: running job 2\n");
    fflush(stdout);

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0)
        _exit(3);

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
        printf("job 2 exit status %d\n", WEXITSTATUS(status));

    return 0;
}
```

6. Compilation and Execution in Linux

Save the program as process.c.

Open the Ubuntu terminal and execute:

Step 1: Compile

gcc process.c -o process

Step 2: Run

./process

Expected Output

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explanation:

- fork() creates the child process.
- execlp() executes the expr command, which calculates 6 × 7 = 42.
- waitpid() waits for the child process to finish.
- WEXITSTATUS(status) retrieves the child's exit code.
- Exit status 0 indicates success, while 3 indicates the second child terminated with code 3.

7. Why stdout is flushed before fork()

fflush(stdout) clears the standard output buffer before creating a child process.

- Before fork(), the parent may have output waiting in its buffer.
- fork() duplicates the parent's memory, including the output buffer.
- Without flushing, both parent and child may print the same buffered text.
- Flushing ensures the previous message is printed only once.

Experiment: Remove second fflush()

Make these two changes:

1. Remove the second fflush(stdout);.
2. Change _exit(3) to exit(3).

Redirect the output to a file:

```
gcc process.c -o process
./process > output.txt
cat output.txt
```

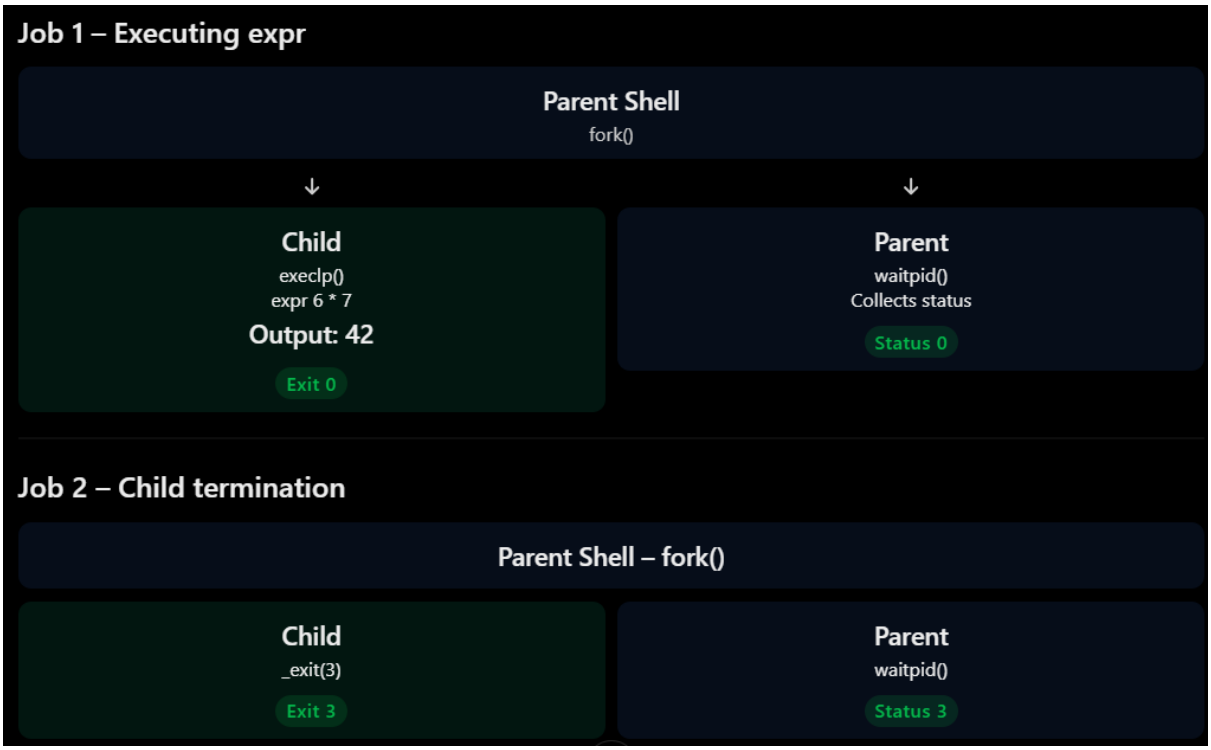
Why does "shell: running job 2" appear twice?

- When stdout is redirected to a file, it is normally fully buffered.
- The second message remains in the output buffer.
- fork() duplicates this buffer into the child process.
- exit(3) flushes the child's copied buffer.
- The parent also flushes its own copy when it exits.
- Therefore, the same message appears twice in the file.

Important difference:

Function	Behaviour
exit()	Flushes standard I/O buffers and terminates the process.
_exit()	Terminates immediately without flushing standard I/O buffers.

Process Diagram:



9. Alternative Concept and Complexity Analysis

An alternative to fork() followed by exec() is posix_spawn().

A. Using posix_spawn()

- posix_spawn() creates a new process and starts the required program.
- It combines process creation and program execution.
- It can reduce overhead compared with a traditional fork() and exec() sequence.
- It supports file redirection and other process attributes.

Example:

```
#include <spawn.h>
#include <sys/wait.h>
#include <stdio.h>

extern char **environ;

int main()
{
   pid_t pid;
```

```
    int status;

    char *args[] = {"expr", "6", "*", "7", NULL};

    posix_spawn(&pid, "/usr/bin/expr",
            NULL, NULL, args, environ);

    waitpid(pid, &status, 0);

    printf("Exit status: %d\n", WEXITSTATUS(status));

    return 0;
}
```

B. Using Threads

- Threads share the same memory space of a process.
- They can execute multiple tasks concurrently.
- Thread creation is generally faster than process creation.
- However, threads do not provide the same memory isolation as separate processes.
- A thread cannot independently call exec() to replace only itself; exec() replaces the entire process.

C. Comparison

Factor	**fork() + exec()**	**posix_spawn()**	**Threads**
Startup time	Higher overhead	Often lower overhead	Usually lowest
Memory	Copy-on-write	Often optimized	Shared memory
Isolation	High	High	Low
Reliability	Good	Good	Depends on synchronization
Best use	Independent programs	Starting external commands	Shared-memory tasks

D. Evaluation for 1,000 Commands

If a database server starts 1,000 commands:

1. fork() + exec(): Requires approximately 1,000 process creations and 1,000 program executions. This increases system-call and process-management overhead.
2. Memory: Copy-on-write avoids immediately copying all parent memory. Only modified pages need separate physical copies. After exec(), the child receives a new program address space.
3. posix_spawn(): Can reduce process creation overhead through operating-system optimizations. It still creates separate processes.

4. Threads: Require less memory because threads share the same address space, but shared data can cause race conditions.
5. Overall: posix_spawn() is suitable for launching many independent external commands, while threads are useful for tasks that need shared data.

10. Conclusion

1. Process control system calls help the shell manage programs efficiently.
2. fork(), exec(), waitpid() and exit() perform important process management operations.
3. Zombie and orphan processes are handled by the operating system.
4. Flushing stdout before fork() prevents duplicate buffered output.
5. posix_spawn() provides an alternative for starting independent commands, while threads offer faster shared-memory execution with less isolation.

Result: The C program successfully demonstrates process creation, command execution, child termination and exit-status collection using Linux system calls.